

ejercicio 1.1:

The screenshot shows the Visual Studio Code editor with a C++ file named `main.cpp` open. The code defines a `Complejo` class with a constructor and an overloaded `+` operator. The `main` function creates two complex numbers, `a` and `b`, and prints the result of their sum using the overloaded operator.

```
1 #include <iostream>
2 class Complejo {
3 public:
4     double re, im;
5
6     // Constructor
7     Complejo(double r, double i) : re(r), im(i) {}
8
9     // Sobrecarga del operador +
10    Complejo operator+(const Complejo& otro) const {
11        return Complejo(re + otro.re, im + otro.im);
12    }
13 };
14
15 int main() {
16     // Creación de los objetos Complejo
17     Complejo a(1.0, 2.0);
18     Complejo b(3.0, -1.0);
19
20     // Suma usando el operador sobrecargado
21     Complejo resultado = a + b;
22
23     // Mostrar el resultado en el formato (re, im)
24     std::cout << "(" << resultado.re << ", " << resultado.im << ")" << std::endl;
25
26     return 0;
27 }
```

The terminal at the bottom shows the command `PS D:\Sesion_3\ejercicios_resueltos>` and the output `(4, 1)`.

ejercicio 1.2:

The screenshot shows the Visual Studio Code editor with a C++ file named `main.cpp` open. The code defines a `Complejo` class with a constructor and an overloaded `+` operator. The `main` function creates two complex numbers, `x` and `y`, and prints the result of their sum using the overloaded operator.

```
1 // complejo_suma.cpp
2 #include <iostream>
3 using namespace std;
4
5 class Complejo {
6 public:
7     double re, im;
8
9     Complejo(double r, double i) : re(r), im(i) {}
10
11    Complejo operator+(const Complejo& otro) {
12        // COMPLETADO: Se suman los componentes y se devuelve un nuevo objeto
13        return Complejo(re + otro.re, im + otro.im);
14    }
15 };
16
17 int main() {
18     Complejo x(0.5, 0.5), y(1.5, 2.0);
19     Complejo z = x + y;
20
21     // Esto imprimirá en la consola: (2, 2.5)
22     cout << "(" << z.re << ", " << z.im << ")" << endl;
23
24     return 0;
25 }
```

The terminal at the bottom shows the command `PS D:\Sesion_3\ejercicios_resueltos> D:\Sesion_3\ejercicios_resueltos\main.exe` and the output `(2, 2.5)`.

ejercicio 1.3:

The screenshot shows the Visual Studio Code editor with a C++ file named `main.cpp`. The code defines a `Complejo` class with a constructor and an overloaded `operator+`. The `main` function creates two complex numbers, `x` and `y`, and prints the result of their addition. The terminal shows the output: `(4, 6)`. The status bar at the bottom indicates the file is at line 25, column 2, with a UTF-8 encoding and CRLF line endings.

```
1 // complejo_bug.cpp
2 #include <iostream>
3 using namespace std;
4
5 class Complejo {
6 public:
7     double re, im;
8
9     Complejo(double r, double i) : re(r), im(i) {}
10
11     // SOLUCIÓN: Firma corregida con referencias, const y lógica matemática correcta
12     Complejo operator+(const Complejo& otro) const {
13         return Complejo(re + otro.re, im + otro.im);
14     }
15 };
16
17 int main() {
18     Complejo x(2.5, 4.0), y(1.5, 2.0);
19     Complejo z = x + y;
20
21     // Ahora la salida será correcta: (4, 6)
22     cout << "(" << z.re << ", " << z.im << ")" << endl;
23
24     return 0;
25 }
```

ejercicio 2.1:

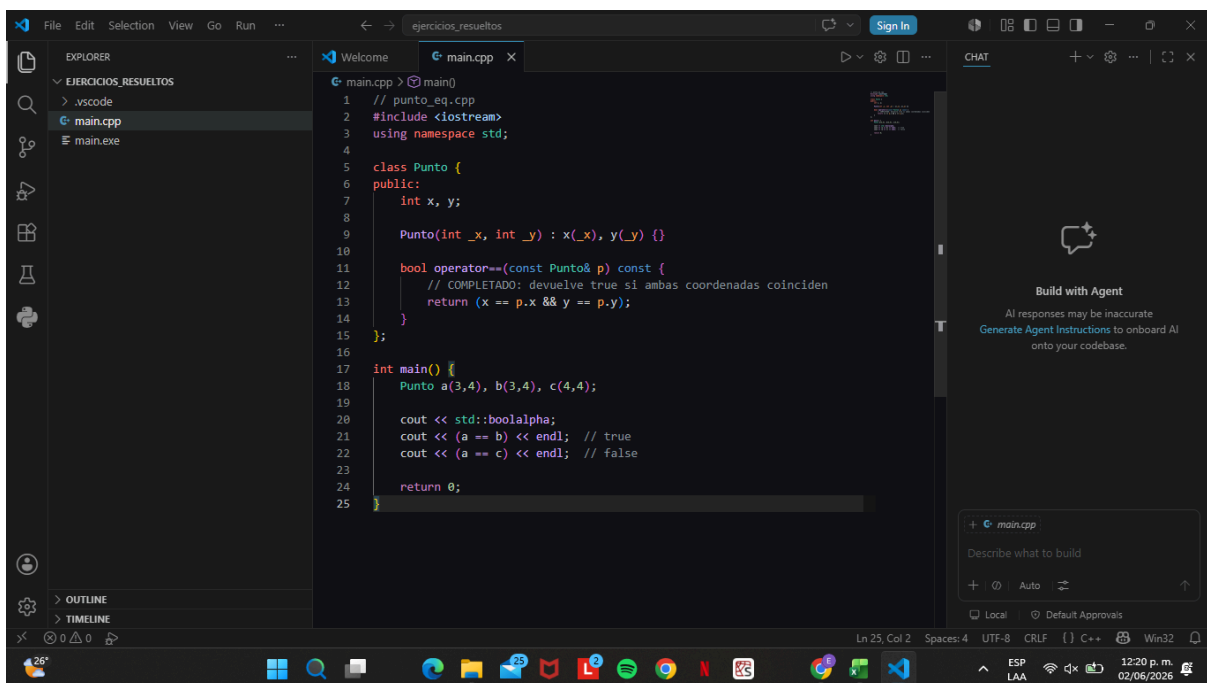
The screenshot shows the Visual Studio Code editor with a C++ file named `main.cpp`. The code defines a `Punto` class with a constructor and an overloaded `operator==`. The `main` function creates three points, `p1`, `p2`, and `p3`, and prints the results of comparisons using `boolalpha`. The terminal shows the output: `Comparando (5,5) con (5,5): true` and `Comparando (5,5) con (5,6): false`. The status bar at the bottom indicates the file is at line 29, column 2, with a UTF-8 encoding and CRLF line endings.

```
1 #include <iostream>
2 using namespace std;
3
4 class Punto {
5 public:
6     int x, y;
7
8     // Constructor
9     Punto(int x, int y) : x(x), y(y) {}
10
11     // Sobrecarga del operador ==
12     bool operator==(const Punto& otro) const {
13         return (x == otro.x && y == otro.y);
14     }
15 };
16
17 int main() {
18     Punto p1(5, 5);
19     Punto p2(5, 5);
20     Punto p3(5, 6);
21
22     // boolalpha hace que imprima "true" o "false" en lugar de "1" o "0"
23     cout << boolalpha;
24
25     cout << "Comparando (5,5) con (5,5): " << (p1 == p2) << endl; // true
26     cout << "Comparando (5,5) con (5,6): " << (p1 == p3) << endl; // false
27
28     return 0;
29 }
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\Sesion_3\ejercicios_resueltos> D:\Sesion_3\ejercicios_resueltos\main.exe
Comparando (5,5) con (5,5): true
Comparando (5,5) con (5,6): false
PS D:\Sesion_3\ejercicios_resueltos> |
```

ejercicios 2.2:



The screenshot shows the Visual Studio Code interface with a C++ file named `main.cpp` open. The code defines a `Punto` class with a constructor and an equality operator. The `main` function creates three points and compares them.

```
1 // punto_eq.cpp
2 #include <iostream>
3 using namespace std;
4
5 class Punto {
6 public:
7     int x, y;
8
9     Punto(int _x, int _y) : x(_x), y(_y) {}
10
11     bool operator==(const Punto& p) const {
12         // COMPLETADO: devuelve true si ambas coordenadas coinciden
13         return (x == p.x && y == p.y);
14     }
15 };
16
17 int main() {
18     Punto a(3,4), b(3,4), c(4,4);
19
20     cout << std::boolalpha;
21     cout << (a == b) << endl; // true
22     cout << (a == c) << endl; // false
23
24     return 0;
25 }
```

```
PS D:\Sesion_3\ejercicios_resueltos> D:\Sesion_3\ejercicios_resueltos\main.exe
true
false
PS D:\Sesion_3\ejercicios_resueltos> |
```

ejercicio 2.3:

```
1 // punto_eq_bug.cpp
2 #include <iostream>
3 using namespace std;
4
5 class Punto {
6 public:
7     int x, y;
8
9     Punto(int _x, int _y) : x(_x), y(_y) {}
10
11     // CORREGIDO: La firma ahora usa const Punto& y el método es const
12     bool operator==(const Punto& p) const {
13         // CORREGIDO: Se cambió '=' por '==' en la comparación de 'y'
14         return x == p.x && y == p.y;
15     }
16 };
17
18 int main() {
19     Punto a(1, 2), b(1, 2), c(2, 2);
20
21     cout << std::boolalpha;
22     cout << "a == b: " << (a == b) << endl; // true
23     cout << "a == c: " << (a == c) << endl; // false
24
25     return 0;
26 }
```

PS D:\Sesion_3\ejercicios_resueltos> D:\Sesion_3\ejercicios_resueltos\main.exe

a == b: true
a == c: false

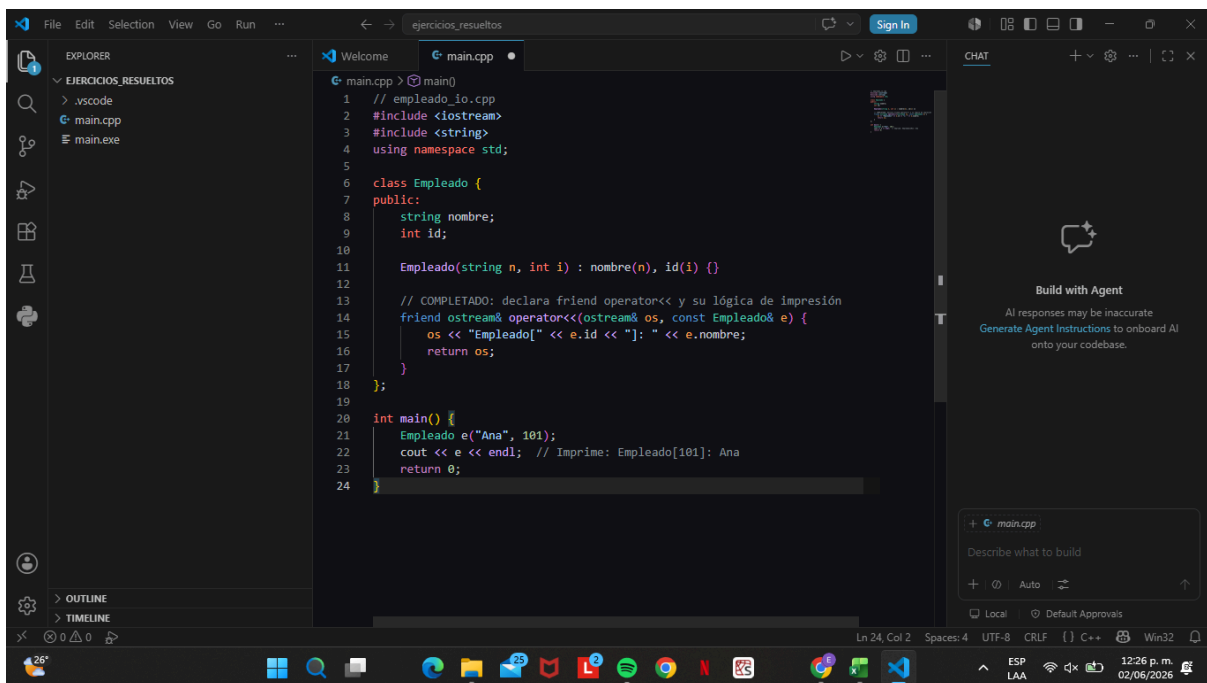
ejercicio 3.1:

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class Empleado {
6 public:
7     string nombre;
8     int id;
9
10    // Constructor
11    Empleado(string n, int i) : nombre(n), id(i) {}
12
13    // Sobrecarga del operador << usando friend
14    friend ostream& operator<<(ostream& os, const Empleado& e) {
15        os << "Empleado[" << e.id << "]: " << e.nombre;
16        return os;
17    }
18 };
19
20 int main() {
21     Empleado e1("Carlos", 102);
22     Empleado e2("Laura", 103);
23
24     // Mostramos a los dos empleados
25     cout << e1 << endl;
26     cout << e2 << endl;
27
28     return 0;
29 }
```

Ln 29, Col 2 Spaces: 4 UTF-8 CRLF {} C++ Win32

```
PS D:\Sesion_3\ejercicios_resueltos> D:\Sesion_3\ejercicios_resueltos\main.exe
Empleado[102]: Carlos
Empleado[103]: Laura
PS D:\Sesion_3\ejercicios_resueltos> 
```

ejercicio 3.2:



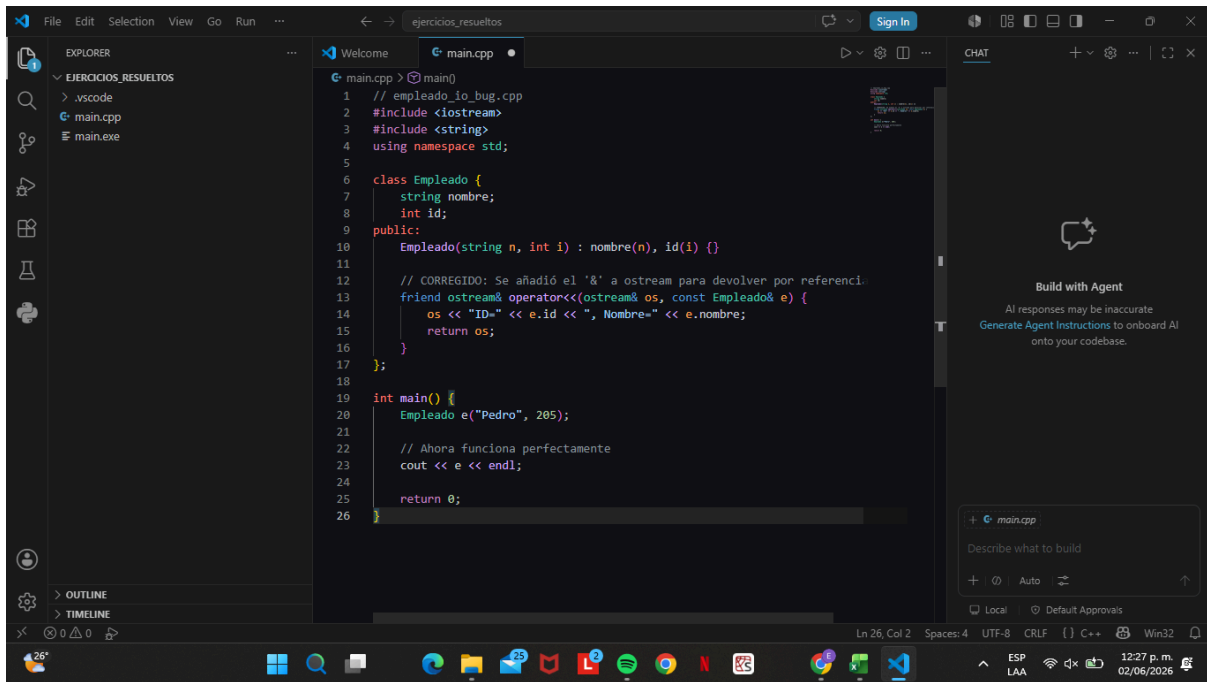
The screenshot shows the Visual Studio Code editor with the file explorer on the left displaying the project structure: `EJERCICIOS_RESUELTOS`, `.vscode`, `main.cpp`, and `main.exe`. The main editor window shows the code for `main.cpp` with the following content:

```
1 // empleado_io.cpp
2 #include <iostream>
3 #include <string>
4 using namespace std;
5
6 class Empleado {
7 public:
8     string nombre;
9     int id;
10
11     Empleado(string n, int i) : nombre(n), id(i) {}
12
13     // COMPLETADO: declara friend operator<< y su lógica de impresión
14     friend ostream& operator<<(ostream& os, const Empleado& e) {
15         os << "Empleado[" << e.id << "]: " << e.nombre;
16         return os;
17     }
18 };
19
20 int main() {
21     Empleado e("Ana", 101);
22     cout << e << endl; // Imprime: Empleado[101]: Ana
23     return 0;
24 }
```

On the right side, there is a 'CHAT' panel with a 'Build with Agent' section, which includes a warning: 'AI responses may be inaccurate' and a button to 'Generate Agent Instructions to onboard AI onto your codebase.' Below this, there is a section for 'main.cpp' with a prompt 'Describe what to build' and buttons for '+', 'Auto', and a refresh icon.

```
PS D:\Sesion_3\ejercicios_resueltos> D:\Sesion_3\ejercicios_resueltos\main.exe
Empleado[101]: Ana
PS D:\Sesion_3\ejercicios_resueltos> 
```

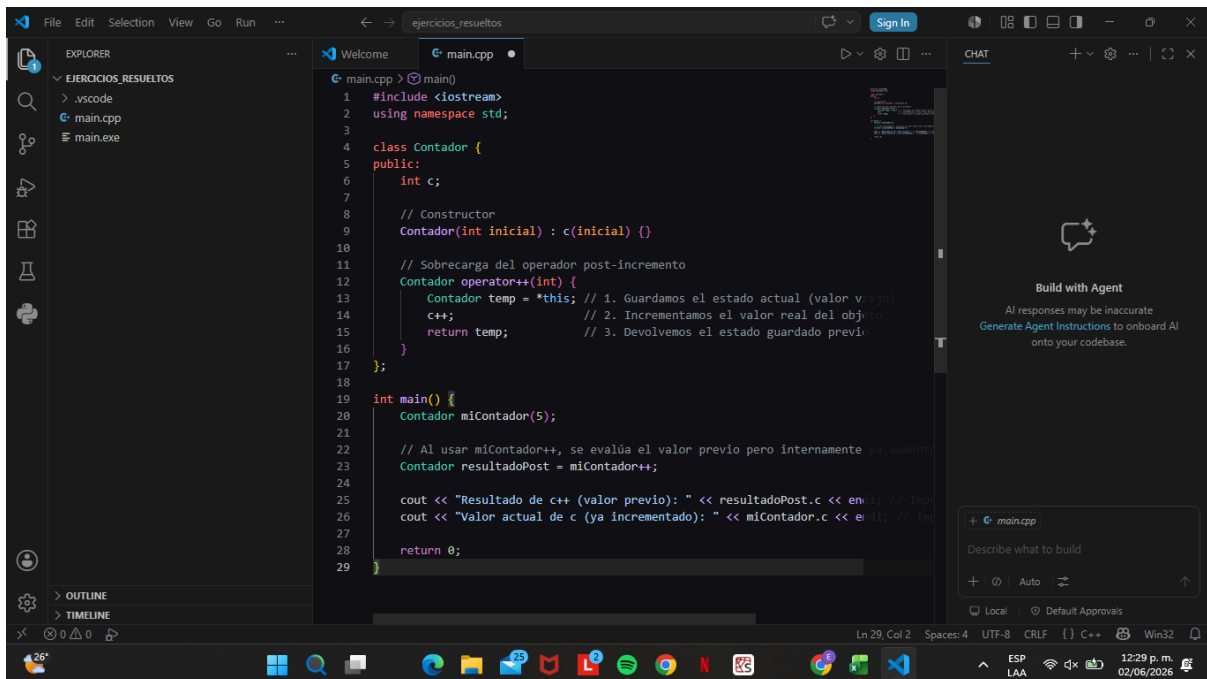
ejercicio 3.3:



```
1 // empleado_io_bug.cpp
2 #include <iostream>
3 #include <string>
4 using namespace std;
5
6 class Empleado {
7     string nombre;
8     int id;
9 public:
10     Empleado(string n, int i) : nombre(n), id(i) {}
11
12     // CORREGIDO: Se añadió el '&' a ostream para devolver por referencia
13     friend ostream& operator<<(ostream& os, const Empleado& e) {
14         os << "ID=" << e.id << ", Nombre=" << e.nombre;
15         return os;
16     }
17 };
18
19 int main() {
20     Empleado e("Pedro", 205);
21
22     // Ahora funciona perfectamente
23     cout << e << endl;
24
25     return 0;
26 }
```

```
PS D:\Sesion_3\ejercicios_resueltos> D:\Sesion_3\ejercicios_resueltos\main.exe
ID=205, Nombre=Pedro
PS D:\Sesion_3\ejercicios_resueltos> █
```

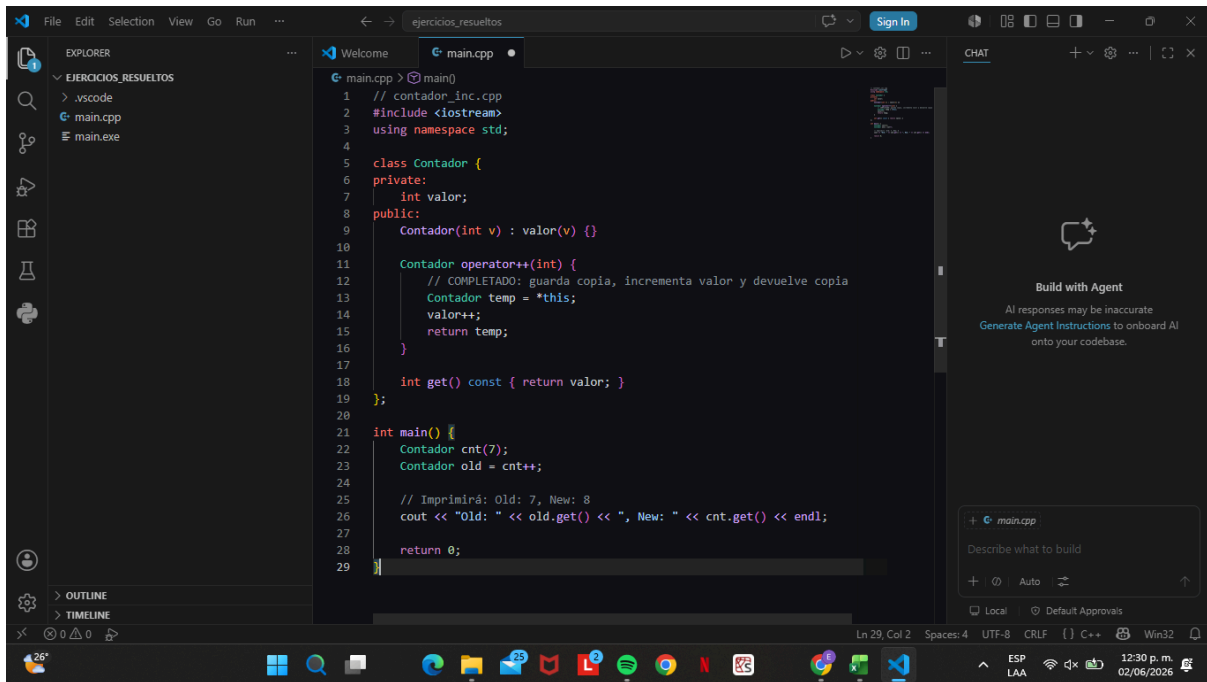
ejercicio 4.1:



```
1 #include <iostream>
2 using namespace std;
3
4 class Contador {
5 public:
6     int c;
7
8     // Constructor
9     Contador(int inicial) : c(inicial) {}
10
11     // Sobrecarga del operador post-incremento
12     Contador operator++(int) {
13         Contador temp = *this; // 1. Guardamos el estado actual (valor previo)
14         c++; // 2. Incrementamos el valor real del objeto
15         return temp; // 3. Devolvemos el estado guardado previamente
16     }
17 };
18
19 int main() {
20     Contador miContador(5);
21
22     // Al usar miContador++, se evalúa el valor previo pero internamente ya aumenta
23     Contador resultadoPost = miContador++;
24
25     cout << "Resultado de c++ (valor previo): " << resultadoPost.c << endl; // 5
26     cout << "Valor actual de c (ya incrementado): " << miContador.c << endl; // 6
27
28     return 0;
29 }
```

```
PS D:\Sesion_3\ejercicios_resueltos> D:\Sesion_3\ejercicios_resueltos\main.exe
Resultado de c++ (valor previo): 5
Valor actual de c (ya incrementado): 6
PS D:\Sesion_3\ejercicios_resueltos> █
```

ejercicio 4.2:



The screenshot shows the Visual Studio Code editor with a C++ file named `main.cpp`. The code defines a `Contador` class with a private `int` member `valor`. The `Contador` class has a constructor `Contador(int v) : valor(v) {}` and a public method `Contador operator++(int)`. The `operator++` method is implemented as follows:

```
Contador operator++(int) {  
    // COMPLETADO: guarda copia, incrementa valor y devuelve copia  
    Contador temp = *this;  
    valor++;  
    return temp;  
}
```

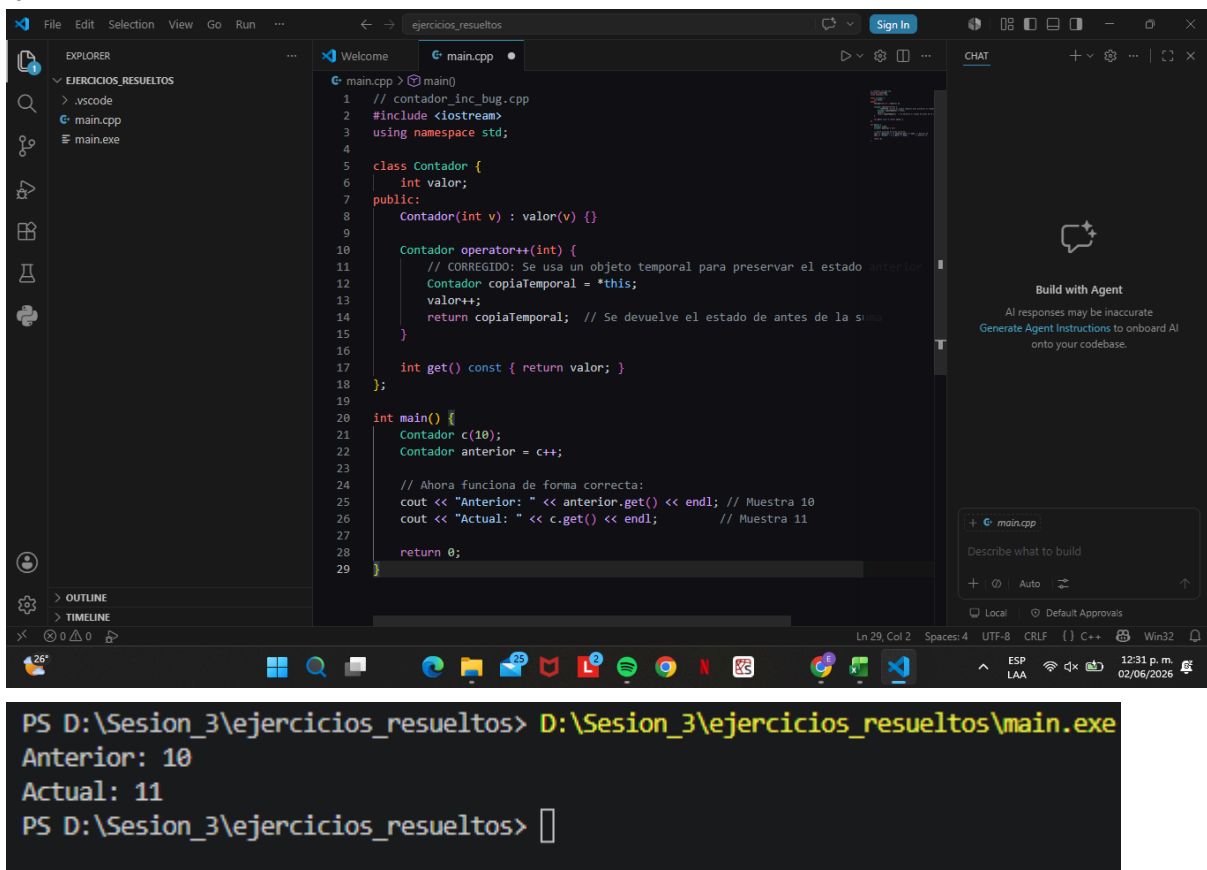
The `main` function creates a `Contador` object `cnt` with the value 7, increments it using the `operator++` method, and prints the old and new values:

```
int main() {  
    Contador cnt(7);  
    Contador old = cnt++;  
  
    // Imprimirá: Old: 7, New: 8  
    cout << "Old: " << old.get() << ", New: " << cnt.get() << endl;  
  
    return 0;  
}
```

The output of the program is shown in the terminal window at the bottom:

```
PS D:\Sesion_3\ejercicios_resueltos> D:\Sesion_3\ejercicios_resueltos\main.exe  
Old: 7, New: 8  
PS D:\Sesion_3\ejercicios_resueltos>
```

ejercicio 4.3:



The screenshot shows the Visual Studio Code editor with a C++ file named `main.cpp`. The code defines a `Contador` class with a private `int` member `valor`. The `Contador` class has a constructor `Contador(int v) : valor(v) {}` and a public method `Contador operator++(int)`. The `operator++` method is implemented as follows:

```
Contador operator++(int) {  
    // CORREGIDO: Se usa un objeto temporal para preservar el estado anterior  
    Contador copiaTemporal = *this;  
    valor++;  
    return copiaTemporal; // Se devuelve el estado de antes de la suma  
}
```

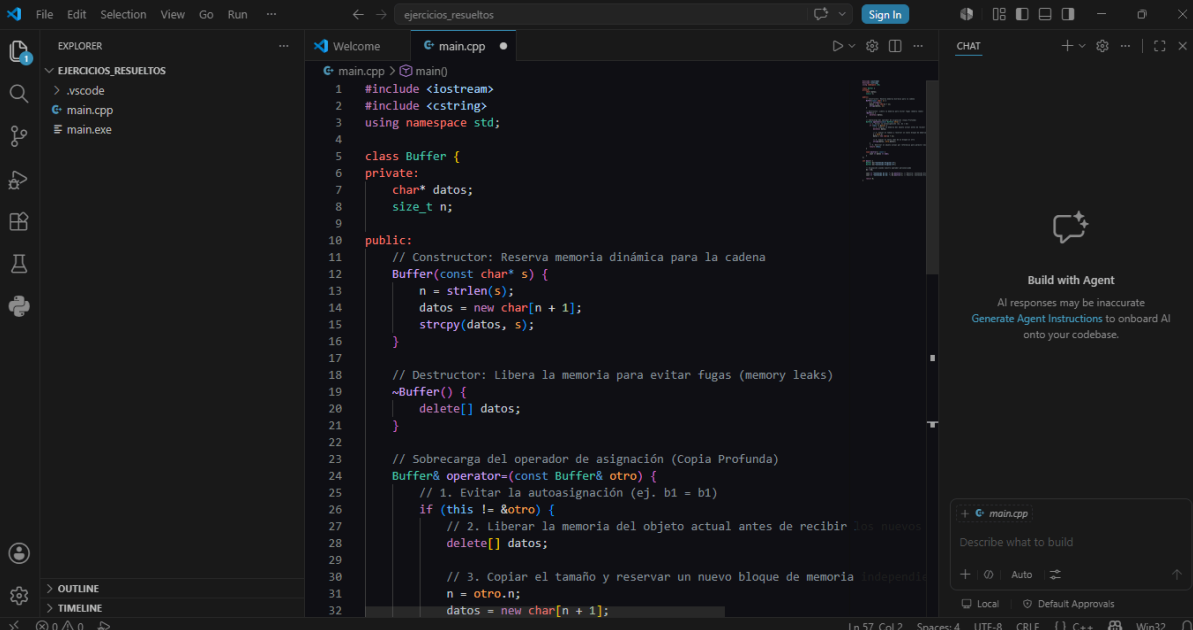
The `main` function creates a `Contador` object `c` with the value 10, increments it using the `operator++` method, and prints the old and new values:

```
int main() {  
    Contador c(10);  
    Contador anterior = c++;  
  
    // Ahora funciona de forma correcta:  
    cout << "Anterior: " << anterior.get() << endl; // Muestra 10  
    cout << "Actual: " << c.get() << endl; // Muestra 11  
  
    return 0;  
}
```

The output of the program is shown in the terminal window at the bottom:

```
PS D:\Sesion_3\ejercicios_resueltos> D:\Sesion_3\ejercicios_resueltos\main.exe  
Anterior: 10  
Actual: 11  
PS D:\Sesion_3\ejercicios_resueltos>
```

ejercicio 5.1:

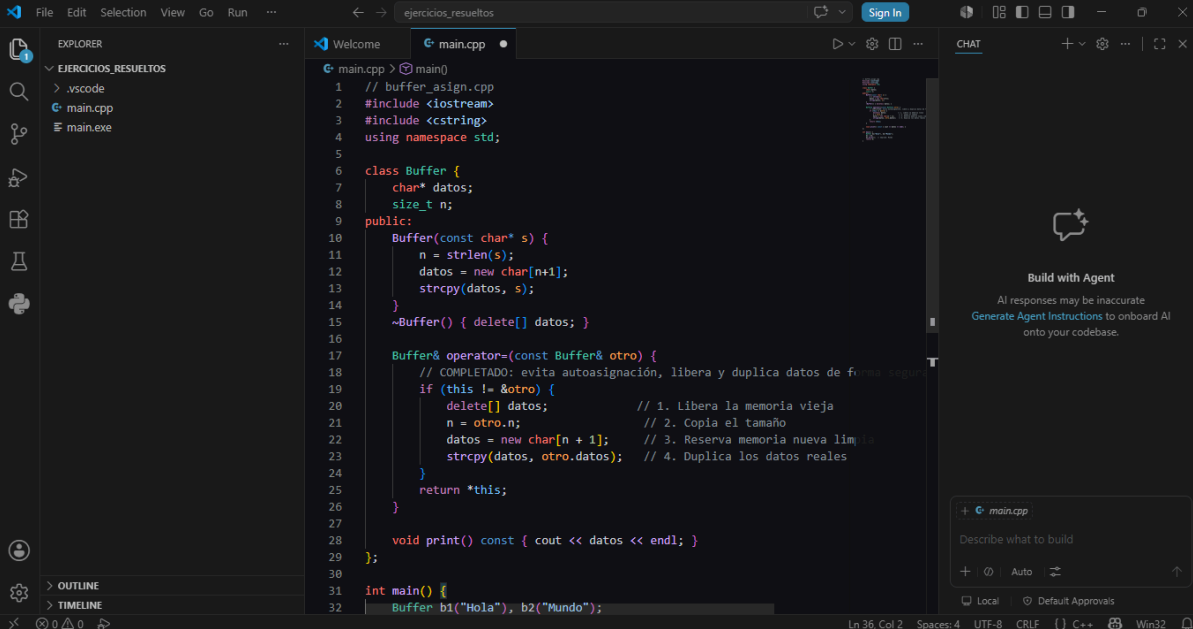


The screenshot shows the Visual Studio Code editor with a C++ file named `main.cpp`. The code defines a `Buffer` class with a private member `char* datos` and `size_t n`. The public methods include a constructor `Buffer(const char* s)` that dynamically allocates memory and copies the string, a destructor `~Buffer()` that frees the memory, and an overloaded assignment operator `Buffer& operator=(const Buffer& otro)` that implements a deep copy. The `main` function is partially visible at the bottom.

```
1 #include <iostream>
2 #include <cstring>
3 using namespace std;
4
5 class Buffer {
6 private:
7     char* datos;
8     size_t n;
9
10 public:
11     // Constructor: Reserva memoria dinámica para la cadena
12     Buffer(const char* s) {
13         n = strlen(s);
14         datos = new char[n + 1];
15         strcpy(datos, s);
16     }
17
18     // Destructor: Libera la memoria para evitar fugas (memory leaks)
19     ~Buffer() {
20         delete[] datos;
21     }
22
23     // Sobrecarga del operador de asignación (Copia Profunda)
24     Buffer& operator=(const Buffer& otro) {
25         // 1. Evitar la autoasignación (ej. b1 = b1)
26         if (this != &otro) {
27             // 2. Liberar la memoria del objeto actual antes de recibir los nuevos
28             delete[] datos;
29
30             // 3. Copiar el tamaño y reservar un nuevo bloque de memoria independiente
31             n = otro.n;
32             datos = new char[n + 1];
```

```
PS D:\Sesion_3\ejercicios_resueltos> D:\Sesion_3\ejercicios_resueltos\main.exe
Contenido de b1: Contenido Original B
Contenido de b2: Contenido Original B
PS D:\Sesion_3\ejercicios_resueltos>
```

ejercicio 5.2:

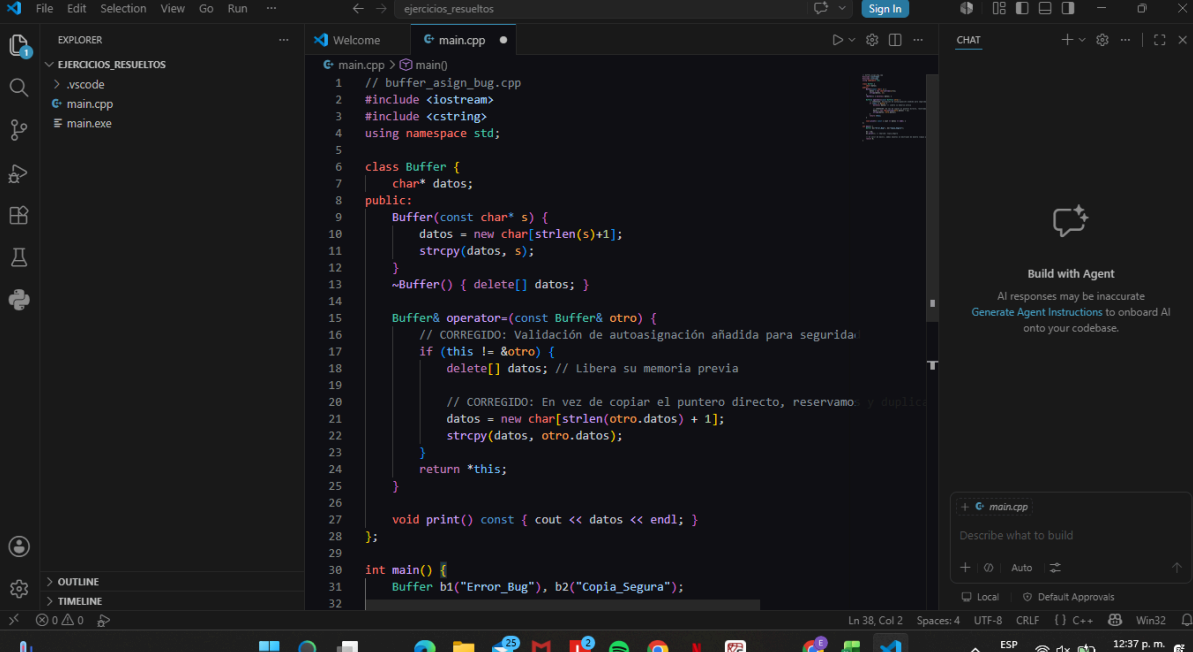


The screenshot shows the Visual Studio Code editor with a C++ file named `main.cpp`. The code defines a `Buffer` class using the copy-and-swap idiom. The constructor `Buffer(const char* s)` allocates memory and copies the string. The destructor `~Buffer()` frees the memory. The assignment operator `Buffer& operator=(const Buffer& otro)` is implemented by swapping the internal data with the other object's data, ensuring a deep copy. The `main` function creates two `Buffer` objects, `b1` and `b2`, and prints their contents.

```
1 // buffer_asign.cpp
2 #include <iostream>
3 #include <cstring>
4 using namespace std;
5
6 class Buffer {
7     char* datos;
8     size_t n;
9
10 public:
11     Buffer(const char* s) {
12         n = strlen(s);
13         datos = new char[n+1];
14         strcpy(datos, s);
15     }
16     ~Buffer() { delete[] datos; }
17
18     // COMPLETADO: evita autoasignación, libera y duplica datos de forma segura
19     if (this != &otro) {
20         delete[] datos; // 1. Libera la memoria vieja
21         n = otro.n; // 2. Copia el tamaño
22         datos = new char[n + 1]; // 3. Reserva memoria nueva limpia
23         strcpy(datos, otro.datos); // 4. Duplica los datos reales
24     }
25     return *this;
26 }
27
28 void print() const { cout << datos << endl; }
29 };
30
31 int main() {
32     Buffer b1("Hola", b2("Mundo");
```

```
PS D:\Sesion_3\ejercicios_resueltos> D:\Sesion_3\ejercicios_resueltos\main.exe
Mundo
PS D:\Sesion_3\ejercicios_resueltos>
```


ejercicio 5.3:



```
1 // buffer_asign_bug.cpp
2 #include <iostream>
3 #include <cstring>
4 using namespace std;
5
6 class Buffer {
7     char* datos;
8 public:
9     Buffer(const char* s) {
10         datos = new char[strlen(s)+1];
11         strcpy(datos, s);
12     }
13     ~Buffer() { delete[] datos; }
14
15     Buffer& operator=(const Buffer& otro) {
16         // CORREGIDO: Validación de autoasignación añadida para seguridad
17         if (this != &otro) {
18             delete[] datos; // Libera su memoria previa
19
20             // CORREGIDO: En vez de copiar el puntero directo, reservamos y duplicamos
21             datos = new char[strlen(otro.datos) + 1];
22             strcpy(datos, otro.datos);
23         }
24         return *this;
25     }
26
27     void print() const { cout << datos << endl; }
28 };
29
30 int main() {
31     Buffer b1("Error_Bug"), b2("Copia_Segura");
32 }
```

```
PS D:\Sesion_3\ejercicios_resueltos> D:\Sesion_3\ejercicios_resueltos\main.exe
Copia_Segura
PS D:\Sesion_3\ejercicios_resueltos> 
```